

Aula 43 - Verbos HTTP e Endpoints de Leitura

Descrição: Esta aula introduz os verbos HTTP no padrão REST, com foco profundo no método GET para leitura de dados. O conteúdo alia teoria sobre parâmetros de busca à construção prática de um endpoint em PHP simulando a Brasil API. Por fim, vamos validar o aprendizado realizando requisições passo a passo através do Postman.

Verbos HTTP e Convenções

1.1 Os Verbos HTTP e o Modelo CRUD

As APIs baseadas no estilo arquitetural REST utilizam os métodos (**verbos**) do protocolo HTTP para determinar qual operação está sendo executada sobre um recurso. O acrônimo CRUD correlaciona-se diretamente com esses métodos:

- **GET** (Read / Leitura): Utilizado estritamente para extrair, listar ou buscar dados de uma API. É um método seguro e idempotente, o que significa que realizar a mesma chamada GET múltiplas vezes não deve alterar o estado do servidor ou do banco de dados.
- **POST** (Create / Criação): Usado para enviar novas informações ao servidor e criar um novo registro.
- **PUT** (Update / Atualização): Destinado à modificação e substituição integral de dados já existentes.
- **DELETE** (Delete / Remoção): Utilizado para apagar um recurso específico do sistema.

1.2 Convenções de URL Design Semântico

Para construir uma Interface Uniforme (um dos 6 pilares do REST), as URLs devem seguir padrões semânticos claros:

- **Use Substantivos no Plural:** A URL aponta para a coleção do recurso (ex: /api/produtos, /api/usuarios).
- **Não use verbos na URL:** Evite caminhos como /api/buscarProduto ou /api/listarProdutos. O verbo da ação já é determinado pelo método HTTP (GET, POST, etc.).

Método HTTP	Endpoint (Rota)	Ação Semântica
GET	/api/produtos	Lista todos os produtos da base de dados.
GET	/api/produtos/:id	Busca os detalhes de um único produto específico pelo ID.
POST	/api/produtos	Cria um novo produto a partir dos dados enviados.
PUT	/api/produtos/:id	Atualiza completamente o produto identificado por aquele ID.
DELETE	/api/produtos/:id	Remove permanentemente o produto indicado por aquele ID.

Endpoints de Leitura – GET: Listar e Buscar

1. O Conceito: O que é o GET?

No modelo CRUD (Create, **Read**, Update, Delete), o método **GET** é o responsável pela **Leitura**. Ele serve exclusivamente para pedir dados ao servidor.

Regras de ouro do GET:

- **É Seguro:** Chamar um GET não deve apagar, alterar ou criar nada no banco de dados.
- **Endpoint Semântico:** A URL deve indicar o recurso no plural.
 - **Exemplo da Brasil API:** <https://brasilapi.com.br/api/pix/v1/participants> (Lista os participantes do PIX).

2. Path Parameters vs. Query Strings

Para os endpoints de leitura (GET), existem duas formas principais de enviar informações complementares na URL para buscar dados específicos:

2.1 Path Parameters (Parâmetros de Caminho)

- **Definição:** São variáveis incorporadas diretamente na estrutura da URL (representados na documentação por `:id` ou `{id}`).
- **Uso:** Identificar um recurso específico e obrigatório dentro de uma coleção.
- **Exemplo:** Na rota `/api/produtos/5`, o número `5` é um *path parameter* indicando que queremos ler especificamente o produto com ID igual a 5.

2.2 Query Strings / Query Parameters (Parâmetros de Busca)

- **Definição:** São parâmetros opcionais anexados ao final da URL após o caractere de interrogação (?) e estruturados no formato `chave=valor`. Múltiplos parâmetros são separados pelo caractere `&`.
- **Uso:** Filtrar, ordenar ou paginar uma lista de recursos existentes sem alterar o endpoint base.
- **Exemplo:** Na rota `/api/produtos?categoria=eletronicos&preco_max=1500`, estamos requisitando a listagem geral, porém filtrando apenas itens da categoria 'eletrônicos' e com preço abaixo de 1500.

Implementando nossa própria "Brasil API - PIX" em PHP

Para que vocês entendam como a API funciona "por baixo dos panos" utilizando parâmetros, vamos criar um script PHP que simula esse endpoint. Ele terá duas funções:

- Retornar **todos** os participantes do PIX.
- Retornar **um** participante específico filtrado via Query String.

Crie um arquivo chamado `pix.php`:

PHP

```
<?php
```

```
// 1. Configura o cabeçalho para dizer ao cliente (Postman/Navegador) que a resposta é em JSON
header("Content-Type: application/json; charset=UTF-8");
```

```
// 2. Simulando o Banco de Dados (Inspirado nos dados reais da Brasil API)
```

```
$participantes = [
    [
        "ispb" => "00000000",
        "nome" => "BANCO DO BRASIL S.A.",
        "nome_reduzido" => "BCO DO BRASIL S.A.",
        "modalidade_participacao" => "PDTA"
    ],
    [
        "ispb" => "00360305",
        "nome" => "CAIXA ECONOMICA FEDERAL",
        "nome_reduzido" => "CAIXA ECONOMICA FEDERAL",
        "modalidade_participacao" => "PDTA"
    ],
    [
        "ispb" => "18236120",
        "nome" => "NU PAGAMENTOS S.A. - INSTITUIÇÃO DE PAGAMENTO",
        "nome_reduzido" => "NUBANK",
        "modalidade_participacao" => "PDTA"
    ]
];
```

```
// 3. Verifica se o método utilizado é realmente o GET
if ($_SERVER['REQUEST_METHOD'] !== 'GET') {
    http_response_code(405); // Method Not Allowed
    echo json_encode(["erro" => "Método não permitido. Utilize apenas GET."]);
    exit;
}

// 4. Captura o parâmetro de busca (Query String), se existir. Ex: pix.php?ispb=18236120
$ispbBuscado = isset($_GET['ispb']) ? $_GET['ispb'] : null;

// 5. Lógica de Resposta
if ($ispbBuscado) {
    // BUSCA ESPECÍFICA (Filtrando pelo ISPB)
    $resultado = array_filter($participantes, function($p) use ($ispbBuscado) {
        return $p['ispb'] === $ispbBuscado;
    });

    if (!empty($resultado)) {
        http_response_code(200); // OK
        // array_values reseta as chaves do array, pegamos o índice [0] para devolver apenas 1 objeto
        echo json_encode(array_values($resultado)[0]);
    } else {
        http_response_code(404); // Not Found
        echo json_encode(["erro" => "Participante PIX não encontrado com o ISPB informado."]);
    }
} else {
    // LISTAGEM GERAL (Nenhum filtro foi passado)
    http_response_code(200); // OK
    echo json_encode($participantes);
}
?>
```

Laboratório Prático: Executando a API no Postman

Etapas 1: Criando o arquivo *pix.php* e preparando o Servidor

1. Certifique-se de que o seu servidor local (como o Apache no XAMPP) está **iniciado** e rodando.
2. Navegue até a pasta pública do seu servidor, a pasta *htdocs* no XAMPP.
3. Dentro dessa pasta, crie um novo arquivo de texto e nomeie-o exatamente como *pix.php*.
4. Abra este arquivo no seu editor de código (como o VS Code), cole todo o código PHP da nossa "Brasil API simulada" (visto no passo anterior) e salve o arquivo.

Etapas 2: Preparando a Requisição no Postman

1. Abra o aplicativo do Postman.
2. Clique no botão "+" (ou em *New > HTTP Request*) para abrir uma nova aba de requisição em branco.
3. Observe a barra superior principal. O menu suspenso à esquerda deve estar configurado com o método **GET**.

Teste 1: Listagem Geral (Nossa API)

Vamos testar se o nosso servidor está devolvendo a lista completa de bancos.

1. Na barra de requisição principal, certifique-se de que a caixa de seleção de métodos (o verbo HTTP) está configurada como **GET** (geralmente é o padrão).

2. Na barra de endereços (ao lado do método), digite a URL local: <http://localhost/pix.php>.
3. Clique no grande botão azul **"Send"** (Enviar) no canto direito.
4. **Analisando o Resultado:**
 - Olhe para a parte inferior da tela (seção *Response*).
 - Na aba **Body**, você deverá ver o formato de texto colorido (JSON) exibindo o array com os 3 bancos que cadastramos no código.
 - No canto superior direito da caixa de resposta, verifique o **Status**. Deve estar verde exibindo **200 OK**.

Teste 2: Consumindo a Brasil API Real

Vamos mostrar aos alunos que o que eles construíram é a exata lógica usada no mundo real.

1. Crie uma nova requisição na coleção (Add Request) e chame de *"Listar Todos - Brasil API"*.
2. Método: **GET**.
3. URL: <https://brasilapi.com.br/api/pix/v1/participants>.
4. Clique em **Send**.
5. **Analisando o Resultado:** A estrutura de retorno no **Body** é exatamente igual à que programamos, mas trará centenas de bancos e instituições financeiras reais.

Teste 3: Busca Específica usando Query Params

A interface do Postman facilita a criação de *Query Strings* sem que você precise digitar o ponto de interrogação (?) manualmente.

1. Logo abaixo da barra de endereço onde você digitou a URL, clique na aba **"Params"**.
2. Você verá uma tabela com as colunas *Key* (Chave) e *Value* (Valor).
3. Na coluna **Key**, digite **ispb**. Na coluna **Value**, digite **18236120** (o código do Nubank).
4. **Observe a mágica:** A sua barra de endereço foi atualizada automaticamente para <http://localhost/pix.php?ispb=18236120>.
5. Clique em **Send** novamente.
6. **Analisando o Resultado:** A aba de resposta inferior agora mostrará apenas **um único bloco JSON**, contendo os dados exclusivos do Nubank. O status continuará **200 OK**.

Teste 4: O Caminho Triste (Erro 404 - Not Found)

Um bom desenvolvedor sempre testa o que acontece quando as coisas dão errado.

1. Mantenha-se na mesma aba e vá até a tabela de **Params**.
2. Altere o valor da chave **ispb** para um número inexistente, como **99999999**.
3. Clique em **Send**.
4. **Analisando o Resultado:**
 - Verifique o código de **Status** à direita. Ele mudará para a cor vermelha/laranja, indicando o erro **404 Not Found**.
 - No **Body** da resposta, você verá a mensagem de erro que programamos no PHP: `{"erro": "Participante PIX não encontrado com o ISPB informado."}`.

Atividade Prática - Após o intervalo

